Collaborative Filtering: Mean Normalization, TensorFlow AutoDiff & the Notation You Keep Tripping On

A notation-first study guide — cost functions decoded, code explained line by line, worked numerical examples

Covers — Notation Decoder · Mean Normalization & the Cost Function · TensorFlow AutoDiff (1D toy problem + full training loop) · Item Similarity · Limitations · Interview Prep · Study Roadmap

0. How This Guide Is Organized

This lecture teaches by contrast: it starts from a broken case (a new user who gets predicted zero stars for everything), fixes it with mean normalization, then simplifies a hard problem (a full collaborative-filtering cost function) down to a 1-parameter toy problem just to teach the TensorFlow syntax, before scaling back up to the real training loop. Because notation is where most of the confusion happens, **Section 1 is a pure notation decoder** you can flip back to any time a symbol trips you up.

- Section 1 — Notation Decoder + the full cost function, explained symbol by symbol.
- Section 2 — Mean normalization: why it's needed, the formulas, a full numeric walkthrough.
- Section 3 — TensorFlow AutoDiff: the 1D toy problem, then the real training loop, code explained line by line.
- Section 4 — Finding related items with learned feature vectors.
- Section 5 — Limitations of collaborative filtering (cold start, side information).
- Section 6 — Interview prep, common pitfalls, and a study roadmap.

1. Notation Decoder

Before anything else, pin down what every symbol in this topic actually means. Almost every point of confusion in collaborative filtering comes from forgetting whether a superscript is a *movie* index or a *user* index. The rule that never changes: **parentheses superscripts index movies or users, subscripts index a feature/parameter dimension.**

Symbol	Read as	Meaning
n_m	“n-sub-m”	Number of movies (items) in the catalog.
n_u	“n-sub-u”	Number of users.
n	“n”	Number of features in each latent vector (embedding size).
Y(i,j)	“Y of i, j”	The actual rating user j gave movie i (only defined when it exists).
r(i,j)	“r of i, j”	Indicator = 1 if user j rated movie i, else 0. This is what lets the cost function ignore missing entries.
$x^{(i)}$	“x, movie i”	Feature vector *for movie i*. The superscript (i) always means “this row belongs to item i.”
$w^{(j)}$, $b^{(j)}$	“w / b, user j”	Parameter vector and bias *for user j*. The superscript (j) always means “this belongs to user j.”
$x_k^{(i)}$	“x sub k, movie i”	The k-th feature (dimension) of movie i’s vector — subscript picks the dimension, superscript picks the movie.
μ_i	“mu sub i”	The average rating of movie i, across only the users who rated it.
$Y_{norm}(i,j)$	“Y-norm of i, j”	Rating after mean normalization: $Y(i,j) - \mu_i$.
λ	“lambda”	Regularization strength — how hard the cost function penalizes large parameter values.
J	“J”	The cost function being minimized by gradient descent / Adam.

The Full Collaborative-Filtering Cost Function

Once you can read every symbol above, this equation stops being scary — it is just three sums stacked together:

$$J(w,b,x) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} \left(w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)}\right)^2 + \frac{\lambda}{2} \sum_{j} \sum_{k} \left(w_k^{(j)}\right)^2 + \frac{\lambda}{2} \sum_{i} \sum_{k} \left(x_k^{(i)}\right)^2$$

Read it in three pieces, left to right:

- **Piece 1 — the fit term:** $\sum_{(i,j):r(i,j)=1}$ means "sum only over the (movie, user) pairs that actually have a rating." For each of those pairs, compare the predicted rating ($w^{(j)} \cdot x^{(i)} + b^{(j)}$) to the real rating $y^{(i,j)}$, square the error, and add it up. This is ordinary squared-error loss.
- **Piece 2 — regularize the users:** $\lambda/2 \cdot \sum_j \sum_k (w_k^{(j)})^2$ penalizes large weight values for every user j across every feature dimension k, so parameters don't grow unbounded.
- **Piece 3 — regularize the movies:** the mirror image of piece 2, but for every movie's feature vector $x^{(i)}$ instead of every user's weights.

Compressed to the shape you should memorize:

$$J = \text{squared error on known ratings} + \frac{\lambda}{2} \sum (w_k^{(j)})^2 + \frac{\lambda}{2} \sum (x_k^{(i)})^2$$

Worked example: say movie 1 was rated by users 1 and 2 only (r(1,1)=1, r(1,2)=1, r(1,3)=r(1,4)=0). The fit term only ever touches $y^{(1,1)}$ and $y^{(1,2)}$ for that row — user 3 and user 4's non-ratings for movie 1 never appear anywhere in J. That is exactly why a user with zero ratings (like Eve, coming up in Section 2) has no fit term touching her parameters at all — only the regularization term does.

2. Mean Normalization

Why It's Needed — The Eve Problem

Suppose a 5th user, **Eve**, joins and hasn't rated a single movie. Look back at Piece 1 of the cost function: since r(i,5)=0 for every movie i, the squared-error term never involves $w^{(5)}$ or $b^{(5)}$ at all. The *only* thing left in J that touches Eve's parameters is the regularization term — and regularization always wants parameters as small as possible. So gradient descent drives $w^{(5)}$ straight to [0, 0], and with $b^{(5)}$ defaulting to 0, her prediction for every single movie collapses:

$$\text{Prediction} = w^{(5)} \cdot x^{(i)} + b^{(5)} = 0 \cdot x^{(i)} + 0 = 0 \text{ stars}$$

Predicting that a brand-new user will hate every movie in the catalog is a useless cold start. Mean normalization fixes this.

The Fix, in Two Formulas

Step A — compute each movie's mean rating (using only the users who rated it):

$$\mu_i = \frac{\sum_{j:\, r(i,j)=1} Y(i,j)}{\sum_{j:\, r(i,j)=1} r(i,j)}$$

Step B — subtract that mean before training, add it back at prediction time:

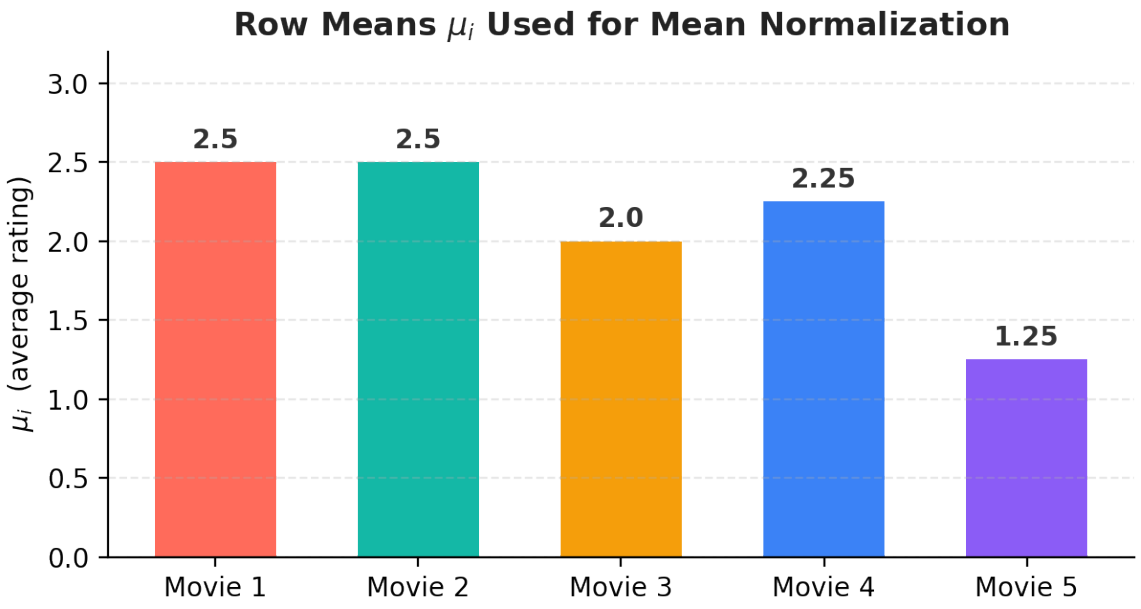
$$Y_{norm}(i,j) = Y(i,j) - \mu_i$$

$$\hat{y}^{(i,j)} = w^{(j)} \cdot x^{(i)} + b^{(j)} + \mu_i$$

Train the model (w, b, x) on Y_{norm} instead of the raw Y. Because the targets are now centered at zero, a user with w=0, b=0 predicts exactly 0 *on the normalized scale* — which, once you add μ_i back in, equals the movie's historical average. That's a far more reasonable cold-start guess than 0 stars.

Full Numeric Walkthrough (5 movies, 5 users, Eve = user 5)

Movie	Ratings by users 1–4	μ_i (row mean)
Movie 1	5, 5, 0, 0	(5+5+0+0)/4 = **2.5**
Movie 2	5, 0 (only 2 raters)	(5+0)/2 = **2.5**
Movie 3	4, 0 (only 2 raters)	(4+0)/2 = **2.0**
Movie 4	mixed ratings	**2.25**
Movie 5	mixed ratings	**1.25**



Two concrete entries to see the subtraction in action: User 1's rating of 5 on Movie 1 becomes 5 – 2.5 = **2.5** in Y_{norm}. User 1's rating of 0 on Movie 4 becomes 0 – 2.25 = –**2.25** (yes, negative — that's expected inside Y_{norm}, it just means "below that movie's average," and gets fixed the moment μ_i is added back).

Now Eve. She still trains to $w^{(5)}$=[0,0], $b^{(5)}$=0 — normalization doesn't change *that*. What changes is the prediction formula, which now adds μ_1 back in:

$$\text{Prediction} = w^{(5)} \cdot x^{(1)} + b^{(5)} + \mu_1 = 0 + 0 + 2.5 = 2.5 \text{ stars}$$

2.5 stars — the movie's average — is a far more useful cold-start guess than 0.

Row vs. Column Normalization

- **Row normalization** (what's used above, normalizing per movie) solves the new-*user* cold start — this is the one that matters in practice.
- **Column normalization** (per user) would help with a brand-new *movie* with zero ratings, but is far less important: a movie with no ratings shouldn't be pushed hard in recommendations anyway until it gathers some.

Takeaway: Mean normalization subtracts each movie's average rating μ_i before training and adds it back before predicting. It doesn't change what a zero-data user learns (w=0, b=0) — it changes what that zero *means*, turning a useless “0 stars for everything” into a sensible “the movie's average rating.”



3. TensorFlow Implementation via AutoDiff

Computing the partial derivatives of the cost function J by hand (with respect to every $w^{(j)}$, $b^{(j)}$, and $x^{(i)}$ simultaneously) is tedious and error-prone. TensorFlow's **Automatic Differentiation (AutoDiff)** computes those derivatives for you — you only have to describe how J is calculated.

*Terminology note: the correct technical term is **Auto Diff**. **AutoGrad** is technically the name of a separate, specific software library, though people often use it loosely to mean the same general idea. Both TensorFlow and PyTorch implement native Auto Diff.*

Step 1: Build Intuition with a 1D Toy Problem

Before touching the real multi-variable collaborative-filtering cost function, strip everything down to one parameter w, a fixed input x=1.0, a target y=1, and bias b=0:

$$J = (wx - 1)^2$$

Starting from w=3.0, running 30 iterations of gradient descent with learning rate α=0.01, here is the code — explained line by line right after:

PYTHON

```python
w = tf.Variable(3.0)
x = 1.0
y = 1.0
alpha = 0.01

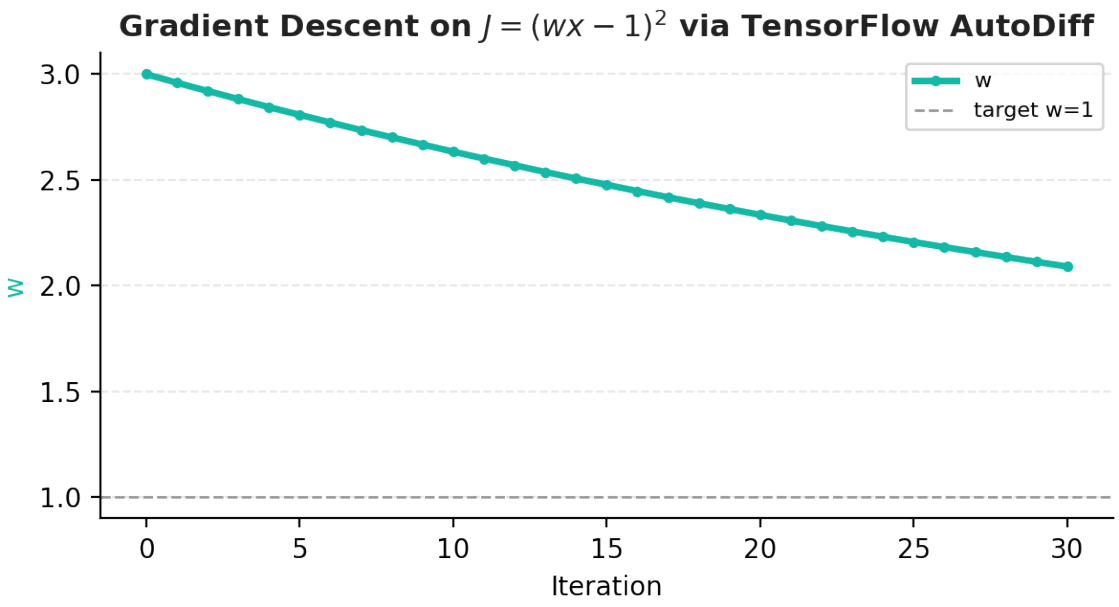
for iteration in range(30):
    with tf.GradientTape() as tape:
        f = w * x
        J = (f - y)**2

    dJdw = tape.gradient(J, w)
    w.assign_sub(alpha * dJdw)
```

- **w = tf.Variable(3.0)** — declares w as a trainable parameter TensorFlow should track. Only tensors wrapped in tf.Variable get gradients recorded; a plain Python float would not.
- **with tf.GradientTape() as tape:** — opens a "recording" context. Every operation performed on a tf.Variable inside this block gets logged onto the tape so it can be differentiated afterward.
- **f = w * x** and **J = (f - y)**2** — this is just forward computation of the cost, but because it happens inside the tape, TensorFlow now has a computation graph it can walk backward through.
- **dJdw = tape.gradient(J, w)** — asks the tape for ∂J/∂w, computed automatically via the chain rule — no calculus written by hand.
- **w.assign_sub(alpha * dJdw)** — the actual gradient-descent update, w ← w – α·dJdw. assign_sub (not a plain `w -= ...`) is required because tf.Variable needs its own special in-place-update methods.

For reference, the derivative TensorFlow is computing automatically at each step is simply:

$$\frac{\partial J}{\partial w} = 2(wx - 1)\,x$$



*By iteration 30, w has converged to roughly **2.09**, still closing in on the true minimizer w=1 (with only 30 steps at α=0.01 it hasn't fully converged yet — more iterations or a larger learning rate would close the gap faster).*

Step 2: Why model.fit() Doesn't Work Here

A natural question once you know Keras: why not just use `tf.keras.Sequential` layers, `model.compile()`, and `model.fit()`? Because collaborative filtering doesn't fit the sequential, feed-forward shape those tools assume. There is no single input flowing through stacked layers to a single output — instead, you are learning **user parameters (w, b) and item parameters (x) simultaneously**, from one matrix-wide cost function that couples every user to every movie at once. That structure needs a custom training loop.

Step 3: The Real Collaborative-Filtering Training Loop

PYTHON

```python
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-1)

for iteration in range(200):
    with tf.GradientTape() as tape:
        # J is computed from the learned parameters (x, w, b),
        # the normalized ratings (y_norm), the indicator matrix (r),
        # user/movie counts (nu, nm), and regularization (lambda)
        J = co_filtering_cost(x, w, b, y_norm, r, nu, nm, lambda_)

    grads = tape.gradient(J, [x, w, b])
    optimizer.apply_gradients(zip(grads, [x, w, b]))
```

- **optimizer = tf.keras.optimizers.Adam(...)** — Adam adapts the learning rate per parameter automatically, which converges much faster than plain gradient descent on a problem this size.
- **co_filtering_cost(...)** — this single call computes exactly the J from Section 1's equation: squared error over rated pairs only (using r as a mask), plus the two regularization sums.
- **tape.gradient(J, [x, w, b])** — the key difference from the toy example: gradients for *three separate variables* (all movie vectors, all user weights, all user biases) are computed together in one call, because they all interact inside the same J.
- **optimizer.apply_gradients(zip(grads, [x, w, b]))** — zips each gradient with its matching variable and lets Adam apply the update to all of them at once.

*This loop is what actually trains on the real-world **MovieLens dataset** (compiled by Harper and Konstan) used in the lab.*

Takeaway: TensorFlow's GradientTape records the forward computation of J, then tape.gradient() walks it backward to get every derivative automatically. The 1D toy problem (one variable, one call to tape.gradient) and the real collaborative-filtering loop (three variables, one call to tape.gradient) use the exact same four-step pattern: declare Variables → record with the tape → get gradients → apply them.

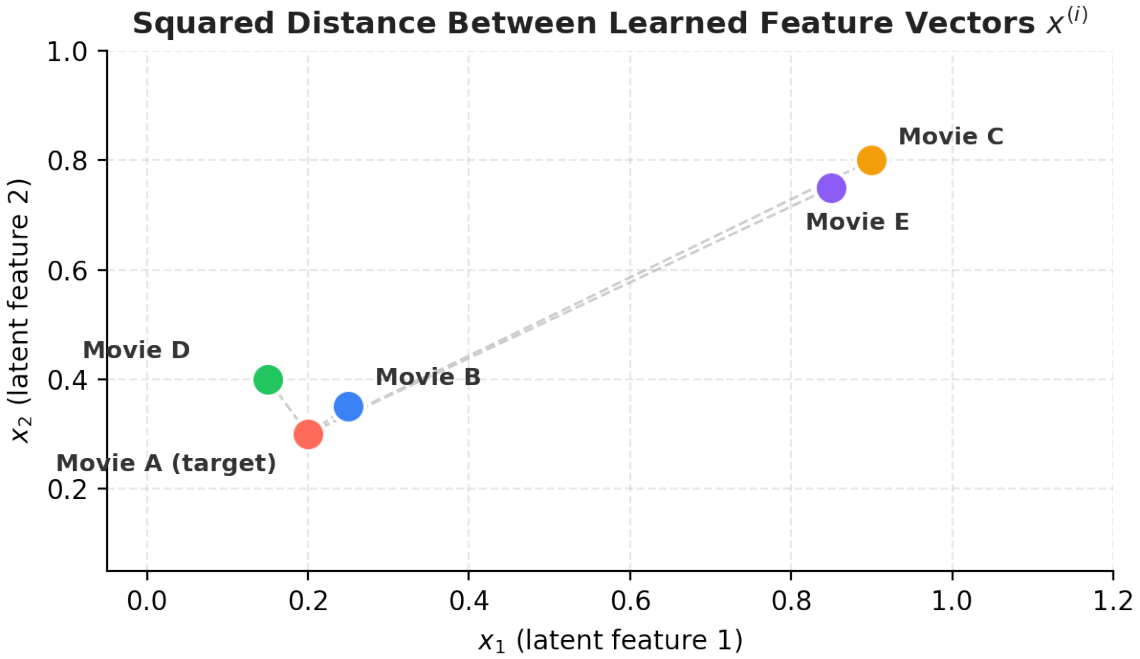
4. Finding Related Items via Learned Features

Once training finishes, each movie i has a learned feature vector $x^{(i)}$. In a toy example you might imagine one dimension meaning “how much action” and another “how much romance,” but in a real trained model the individual dimensions x_1, x_2, …, x_n are highly abstract and don't line up with any single human-readable concept. Taken *together*, though, the full vector acts as a precise mathematical “signature” of the movie.

To find items related to movie i, measure the squared Euclidean distance to every other movie k:

$$\text{Distance}(k, i) = \sum_{l=1}^{n} \left(x_l^{(k)} - x_l^{(i)}\right)^2 = \|x^{(k)} - x^{(i)}\|^2$$

Compute this distance for every $k \neq i$, sort ascending, and keep the smallest handful (5–10) — those are the most mathematically similar items.



Worked example: in the toy plot above, Movie B sits almost on top of Movie A in feature space (a tiny squared distance), while Movie C and Movie E sit far away. A “you might also like” widget for Movie A would surface Movie B first — exactly the mechanism e-commerce sites use for “customers who viewed this item also viewed’’ carousels, just with the item being a product embedding instead of a movie.

Takeaway: Similarity between two items is just squared Euclidean distance between their learned vectors: small distance = similar signature = good recommendation candidate, even though no individual dimension of that vector has an obvious name.

5. Limitations of Collaborative Filtering

1. The Cold Start Problem

- **New items:** a brand-new movie has zero ratings, so there's no data to optimize its feature vector x — it can't be recommended accurately yet.
- **New users:** a brand-new user has rated few or no items. Mean normalization gives a reasonable *baseline* guess, but it still isn't personalized until the user builds up a ratings history.

2. Inability to Leverage Side Information

Collaborative filtering only ever sees the ratings matrix — it is blind to any richer metadata:

Category	Examples
Item side info	Genre, director, cast, production studio, budget
User side info	Age, gender, location, explicit stated preferences
Contextual cues	IP address / location, device type, browser used

One notable finding: users on different browsers (Chrome vs. Firefox vs. Safari vs. Microsoft Edge) show measurably different behavior patterns — browser choice can correlate with preferences, but plain collaborative filtering has no way to use that signal at all.

This is exactly the gap **content-based filtering** closes, by folding side information directly into the model instead of relying purely on the ratings matrix.

> **Takeaway:** Collaborative filtering only knows what it sees in the ratings matrix. No ratings for a movie or user, and no side information ever, means it hits a wall — which is precisely the motivation for content-based filtering.

6. Interview Prep, Common Pitfalls & Study Roadmap

Interview-Style Q&A

Q: *Describe the cold-start problem for new users. Without mean normalization, what does a new user's prediction collapse to, and why?*

Key point: With zero ratings, the squared-error term in J never touches that user's parameters — only regularization does, which drives w to [0,0]. With b defaulting to 0, every prediction becomes exactly 0. Row-based mean normalization shifts the target matrix so a user with w=0, b=0 naturally predicts each movie's historical average μ_i instead.

Real-world application: When a music streaming service launches in a new country, brand-new users have no listening history. Instead of predicting zero interest, the app serves the country's average trending tracks (row mean normalization) until real preferences accumulate.

Q: *Why can't collaborative filtering be trained with plain Keras Sequential layers and model.fit()? What does the custom loop need to do instead?*

Key point: User parameters (w, b) and item parameters (x) are learned simultaneously from one shared, matrix-wide cost function — that doesn't fit a sequential feed-forward architecture. A custom loop records the cost with tf.GradientTape(), calls tape.gradient(J, [x, w, b]) to get all derivatives at once, and applies them with an optimizer like Adam.

Real-world application: Deep reinforcement learning and GANs face the same issue — complex loss functions with multiple interacting components — and rely on custom GradientTape loops for the same reason.

Q: *Can you interpret the individual dimensions of a trained $x^{(i)}$? How do you find similar items?*

Key point: No — learned latent features are abstract and rarely map to human concepts like “romance” or “sci-fi.” But the full vector still captures behavior geometrically: similarity is the squared Euclidean distance $\sum(x_l^{(k)} - x_l^{(i)})^2$, and the smallest distances are the most similar items.

Real-world application: E-commerce platforms compute distances between learned product embeddings to power “customers who viewed this also viewed” carousels.

features هو هنا مش شايفه احياء ال
هو بيقرب بالأرقام وبس

Foundational vs. Supplementary

Foundational (must master)	Supplementary (good context)
• Row-wise mean normalization: subtract μ_i to train, add it back to predict • tf.Variable + tf.GradientTape to record a computation • tape.gradient() + optimizer.apply_gradients() to update parameters • The squared-distance similarity formula	• Row vs. column normalization trade-offs • “Auto Diff” vs. the “AutoGrad” library name • Browser choice correlating with user behavior

Commonly Misunderstood

- **Forgetting the “add back” step:** skip adding μ_i at prediction time and the model can output negative star ratings — a physically impossible result.
- **Overrating column normalization:** learners often assume user-based (column) normalization matters as much as row normalization. In practice row wins, since a brand-new movie with no ratings shouldn't be pushed hard in recommendations anyway.
- **Forgetting tf.Variable:** if w, b, or x aren't declared as tf.Variable, TensorFlow can't track them on the tape, and tape.gradient() silently returns None.

Recommended Next Steps

1. Build a dummy 3×3 ratings matrix with missing values by hand. Compute μ, build Y_{norm}, and manually verify a new user's prediction comes out non-zero.
2. Write a GradientTape script for a different 1D toy problem, e.g. $J=(w-7)^2$, starting from w=tf.Variable(0.0), and watch it converge to 7.0.
3. Write a raw NumPy function that takes one item vector and a matrix of candidates, computes squared Euclidean distance to each, sorts them, and returns the top 3 closest — to confirm you understand similarity search without TensorFlow's help.

Takeaway: Every formula in this guide reduces to the same habit: identify whether a superscript means “this movie” or “this user,” check whether the sum is masked by r(i,j), and remember that mean normalization only ever changes what a zero prediction *means*, never how a zero-data user is trained.